

Regressão Linear Múltipla

Guia de Estudo Passo a Passo + Checklist de Projeto

Baseado no Projeto 3 — DSA

Expectativa de Vida

O que é este material: um roteiro replicável, baseado no projeto que você já executou, para conduzir uma Regressão Linear Múltipla do zero em qualquer novo dataset — da carga dos dados ao deploy do modelo. Cada etapa traz o **que fazer**, o **porquê**, o **código de referência** e, quando existir, a **armadilha comum** que o próprio projeto original evidenciou.

Preparado para Daiane · MBA Data Science, AI & Analytics · USP ESALQ

Sumário

- 1. Checklist Rápido de Início de Projeto
- 2. Fase 1 — Carga e Compreensão dos Dados
- 3. Fase 2 — Análise Exploratória
- 4. Fase 3 — Tratamento de Outliers e Valores Ausentes
- 5. Fase 4 — Engenharia de Atributos
- 6. Fase 5 — Correlação e Multicolinearidade
- 7. Fase 6 — Seleção de Variáveis com Random Forest
- 8. Fase 7 — Pré-Processamento (Split e Padronização)
- 9. Fase 8 — Modelagem: Construindo e Comparando Versões
- 10. Fase 9 — Seleção do Modelo Final
- 11. Fase 10 — Diagnóstico de Resíduos
- 12. Fase 11 — Deploy e Uso em Novos Dados
- 13. Tabela-Resumo: Comparando as 5 Versões do Modelo
- 14. Mapa de Fluxo do Projeto

1. Checklist Rápido de Início de Projeto

Cole este checklist no topo de qualquer novo notebook de Regressão Linear Múltipla

- ☐ Carreguei os dados e conferi shape, tipos e amostra aleatória (.sample())
- ☐ Renomeei colunas e documentei cada variável em um dicionário de dados
- ☐ Explorei a variável-alvo (histograma) e relações com atributos (pairplot)
- ☐ Quantifiquei valores ausentes (contagem e %) por coluna
- ☐ Detectei outliers via IQR nas variáveis mais críticas
- ☐ Decidi e apliquei o tratamento de outliers (remover, capar, transformar) — com justificativa registrada
- ☐ Decidi e apliquei o tratamento de ausentes (mediana/média/drop) — com justificativa registrada
- ☐ Removi ou codifiquei variáveis categóricas não usadas na regressão
- ☐ Criei variáveis derivadas relevantes (engenharia de atributos), se fizer sentido
- ☐ Analisei correlação das preditoras com o alvo (1º filtro)
- ☐ Analisei correlação entre preditoras para eliminar multicolinearidade (2º filtro)
- ☐ Validei a relevância das variáveis com Random Forest (feature_importances_)
- ☐ Separei X e y, fiz o split treino/teste
- ☐ Apliquei StandardScaler — **fit só no treino**, transform em treino e teste
- ☐ Treinei um modelo benchmark simples (Linear Regression)
- ☐ Testei variações com regularização (Ridge, Lasso, ElasticNet)
- ☐ Otimizei hiperparâmetros do melhor candidato (GridSearchCV)
- ☐ Comparei todas as versões por RMSE e R² (treino e teste)
- ☐ Escolhi o modelo final — priorizando simplicidade quando a performance é equivalente
- ☐ Plotei e interpretei os resíduos (vs. valores previstos)
- ☐ Salvei modelo e scaler com joblib
- ☐ Testei o pipeline completo com dados novos (aplicando o mesmo scaler antes de prever)

Fase 1 — Carga e Compreensão dos Dados

Passo 1.1 — Carregar e inspecionar

Por quê: antes de qualquer decisão, é preciso saber o tamanho do dataset, os tipos de cada coluna e ter uma primeira visão dos valores reais (não só o `head()`, que sempre mostra as mesmas linhas).

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn import metrics
import joblib

# Carregando os dados
dados = pd.read_csv('dataset.csv')
dados.shape
dados.sample(10)
dados.dtypes
```

Passo 1.2 — Renomear colunas e criar dicionário de dados

Por quê: nomes de colunas limpos evitam erros de digitação e facilitam o código. O dicionário de dados serve de referência rápida para lembrar o que cada variável significa (essencial em datasets com 20+ colunas).

```
df = pd.DataFrame({'alvo': dados['NomeOriginalDaColuna'],
                  'var1': dados['OutraColuna'],
                  # ... mapeie todas as colunas relevantes
                  })

dicionario = {
    "alvo": "Descrição da variável alvo.",
    "var1": "Descrição da variável 1.",
    # ...
}
```

O que realmente importa: remova, já nesta etapa, colunas sem descrição clara na fonte de dados — elas geram ambiguidade na interpretação depois.

Passo 2.1 — Explorar a variável-alvo e suas relações

Por quê: o histograma mostra a forma da distribuição do alvo (simétrica? enviesada?). O pairplot mostra visualmente se há relação linear entre cada atributo e o alvo, e ajuda a notar outliers "a olho".

```
df['alvo'].hist()

df.describe()                # variáveis numéricas
df.describe(include=['object']) # variáveis categóricas

# Função reutilizável de pairplot em blocos de n gráficos por linha
def get_pairs(data, alvo, atributos, n):
    grupos = [atributos[i:i+n] for i in range(0, len(atributos), n)]
    for linha in grupos:
        sns.pairplot(x_vars=linha, y_vars=alvo, data=data, kind="reg", height=3)

get_pairs(df, ['alvo'], ['var1', 'var2', 'var3'], 3)
```

Fase 3 — Tratamento de Outliers e Valores Ausentes

Passo 3.1 — Quantificar valores ausentes

```
ausentes = df.isnull().sum().sort_values(ascending=False)
ausentes_percent = ausentes[ausentes > 0] / df.shape[0]
print(f'{ausentes_percent * 100} %')
```

Passo 3.2 — Detectar outliers (método IQR)

Por quê: o método do IQR (Intervalo Interquartil) é um critério objetivo e amplamente aceito para marcar valores extremos, sem depender de suposição de normalidade.

```
Q1 = df[atributos].quantile(0.25)
Q3 = df[atributos].quantile(0.75)
IQR = Q3 - Q1

limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

outliers = ((df[atributos] < limite_inferior) | (df[atributos] > limite_superior)).sum()
outliers_resumo = pd.DataFrame({'Outliers': outliers,
                                'Percentual': (outliers / len(df)) * 100})
outliers_resumo[outliers_resumo['Outliers'] > 0]
```

O que realmente importa: $1.5 \times \text{IQR}$ é a convenção mais comum, mas não é regra fixa — sempre combine com inspeção visual (pairplot/boxplot) antes de decidir remover.

Passo 3.3 — Tratar outliers e ausentes (com justificativa registrada)

```
# Remover outliers fora dos limites
df_limpo = df[~((df[atributos] < limite_inferior) |
               (df[atributos] > limite_superior)).any(axis=1)]

# Imputar ausentes pela mediana (robusta a outliers) e depois dropar o restante
def impute_median(coluna):
    return coluna.fillna(coluna.median())

df_limpo.loc[:, 'var1'] = df_limpo['var1'].transform(impute_median)
df_limpo.dropna(inplace=True)

# Remover categóricas que não entrarão na regressão (id, texto livre, etc.)
df_limpo.drop(['coluna_categorica'], axis=1, inplace=True)
```

Por quê mediana, e não média? a mediana é robusta a outliers — um valor extremo não distorce o valor usado para preencher os ausentes, ao contrário da média.

Armadilha comum: decidir o tratamento de outliers/ausentes sem documentar o motivo. Sempre registre a decisão (remover vs. imputar vs. transformar) e a justificativa — isso é parte da qualidade do projeto, não só um detalhe.

Passo 4.1 — Criar variáveis derivadas quando fizer sentido

Por quê: combinar duas variáveis com relação natural pode capturar um efeito conjunto que nenhuma das duas sozinha representa bem.

```
# Exemplo do projeto: combinar IMC e consumo de álcool num indicador de "estilo de vida"
df_limpo['lifestyle'] = df_limpo['bmi'] * df_limpo['alcohol']
```

O que realmente importa: toda variável nova criada deve ser checada quanto a valores ausentes/infinitos resultantes da combinação, antes de seguir.

Fase 5 — Correlação e Multicolinearidade

Passo 5.1 — Primeiro filtro: correlação com o alvo

Por quê: queremos preditoras que realmente se relacionem com o que estamos prevendo. Um limiar (ex: $|\text{corr}| \geq 0.3$) separa candidatas fracas de fortes.

```
def filtrar_e_visualizar_correlacao(df, threshold, drop_column=None):
    corr = df.corr()
    filtro = (abs(corr) >= threshold) & (corr != 1.0)
    df_filtrado = corr.where(filtro).dropna(how='all').dropna(axis=1, how='all')
    if drop_column:
        df_filtrado = df_filtrado.drop(index=drop_column, errors='ignore')\
            .drop(columns=drop_column, errors='ignore')

    plt.figure(figsize=(8, 6))
    sns.heatmap(df_filtrado, annot=True, cmap='coolwarm', center=0)
    plt.show()

filtrar_e_visualizar_correlacao(df_limpo, threshold=0.3)
```

Passo 5.2 — Segundo filtro: multicolinearidade entre preditoras

Por quê: alta correlação *entre preditoras* (não com o alvo) fere uma das suposições da regressão linear e infla a variância dos coeficientes, tornando-os instáveis e difíceis de interpretar.

```
# Threshold mais alto (ex: 0.55) e excluindo o alvo da análise
filtrar_e_visualizar_correlacao(df_limpo, threshold=0.55, drop_column='alvo')

# Ao encontrar pares correlacionados, mantenha só uma das duas variáveis
df_final = df_limpo[['alvo', 'var_mantida_1', 'var_mantida_2', ...]]
```

O que realmente importa: correlação não implica causalidade — duas variáveis correlacionadas podem estar ligadas por um terceiro fator (ex: sazonalidade), não uma causando a outra.

Limitação deste método: a correlação par-a-par só enxerga relações entre duas variáveis por vez. Se quiser um diagnóstico mais completo de multicolinearidade (incluindo combinações de 3+ variáveis), o VIF (Variance Inflation Factor) é o próximo passo natural — vale estudar como complemento.

Fase 6 — Seleção de Variáveis com Random Forest

Passo 6.1 — Validar a relevância das variáveis

Por quê: o Random Forest não exige padronização e lida bem com relações não-lineares, servindo como uma segunda opinião (independente da correlação linear) sobre quais variáveis realmente importam.

```
X = df_final.drop('alvo', axis=1)
y = df_final['alvo']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

modelo_rf = RandomForestRegressor(n_estimators=100, random_state=42)
modelo_rf.fit(X_train, y_train)

importancias = pd.DataFrame({'Variável': X.columns,
                             'Importância': modelo_rf.feature_importances_})\
                .sort_values(by='Importância', ascending=False)
print(importancias)

# Remova as variáveis com importância desprezível
df_final = df_final.drop('variavel_menos_importante', axis=1)
```

O que realmente importa: essa etapa é uma checagem, não uma obrigação — se a análise de correlação já te deixou confiante, ela serve para confirmar (ou contestar) essa escolha antes de seguir para a modelagem final.

Fase 7 — Pré-Processamento (Split e Padronização)

Passo 7.1 — Separar X/y, dividir treino/teste e padronizar

```
X = df_final[['var1', 'var2', 'var3']].values
y = df_final['alvo'].values.reshape(-1, 1)

x_treino, x_teste, y_treino, y_teste = train_test_split(X, y, test_size=0.2, random_state=0)

# Algoritmos de regressão (diferente do Random Forest) precisam de dados padronizados
scaler = StandardScaler()
scaler.fit(x_treino) # fit SOMENTE com treino

x_treino_scaled = scaler.transform(x_treino)
x_teste_scaled = scaler.transform(x_teste)
```

Armadilha comum (destacada no próprio projeto original!): fazer o fit do scaler com os dados de teste, ou esquecer de padronizar o teste antes de prever. Isso causa **vazamento de dados (data leakage)** e previsões erradas. Regra de ouro: **fit só no treino, transform em treino e teste.**

Fase 8 — Modelagem: Construindo e Comparando Versões

Passo 8.1 — Versão 1: Regressão Linear (benchmark)

```
modelo_v1 = LinearRegression()
modelo_v1.fit(x_treino_scaled, y_treino)

y_pred_treino = modelo_v1.predict(x_treino_scaled)
y_pred_teste = modelo_v1.predict(x_teste_scaled)

print('RMSE treino:', np.sqrt(metrics.mean_squared_error(y_treino, y_pred_treino)))
print('R2 treino:', metrics.r2_score(y_treino, y_pred_treino))
print('RMSE teste:', np.sqrt(metrics.mean_squared_error(y_teste, y_pred_teste)))
print('R2 teste:', metrics.r2_score(y_teste, y_pred_teste))
```

Por quê começar pelo modelo mais simples? ele vira sua linha de base (benchmark). Se um modelo mais complexo não superar essa linha de base, não vale a pena usar o mais complexo.

Passo 8.2 — Versões 2, 3 e 4: Regularização (Lasso, Ridge, ElasticNet)

```
# V2 - Lasso (L1): pode zerar coeficientes (seleção de variáveis embutida)
modelo_v2 = Lasso(alpha=1.0)
modelo_v2.fit(x_treino_scaled, y_treino)

# V3 - Ridge (L2): reduz coeficientes sem zerá-los
modelo_v3 = Ridge(alpha=1.0)
modelo_v3.fit(x_treino_scaled, y_treino)

# V4 - ElasticNet: combinação de L1 e L2
modelo_v4 = ElasticNet(alpha=1.0, l1_ratio=0.5)
modelo_v4.fit(x_treino_scaled, y_treino)

# Para cada versão: repita o mesmo bloco de predição e métricas do Passo 8.1
```

Passo 8.3 — Versão 5: Otimização de hiperparâmetros (GridSearchCV)

```
modelo_v5 = Ridge()
parametros = {'alpha': [0.01, 0.1, 1, 10, 100]}

grid_search = GridSearchCV(estimator=modelo_v5,
                           param_grid=parametros,
                           cv=5,
                           scoring='neg_mean_squared_error',
                           verbose=1)

grid_search.fit(x_treino_scaled, y_treino)
melhor_modelo = grid_search.best_estimator_
print('Melhor alpha:', grid_search.best_params_['alpha'])
```

Por quê usar validação cruzada aqui? o cv=5 testa cada valor de alpha em 5 divisões diferentes dos dados de treino, dando uma estimativa mais confiável de qual hiperparâmetro realmente generaliza melhor — em vez de confiar num único split.

Passo 9.1 — Comparar todas as versões

Por quê usar mais de uma métrica? RMSE mostra o tamanho do erro; R^2 mostra a proporção da variância explicada. Usadas juntas, dão um quadro mais completo do que qualquer uma isolada.

```
print('RMSE V1:', np.sqrt(metrics.mean_squared_error(y_teste, y_pred_teste_v1)))
print('RMSE V2:', np.sqrt(metrics.mean_squared_error(y_teste, y_pred_teste_v2)))
# ... repita para V3, V4, V5

print('R2 V1:', metrics.r2_score(y_teste, y_pred_teste_v1))
# ... repita para V3, V4, V5
```

O que realmente importa: quando duas versões têm performance parecida, escolha a **mais simples** (menos hiperparâmetros/complexidade) — ela tende a generalizar melhor e é mais fácil de manter e explicar.

Passo 10.1 — Calcular e plotar os resíduos

Por quê: os resíduos (erro = valor real – valor previsto) revelam se o modelo está violando suposições da regressão linear. Um bom modelo deve ter resíduos espalhados aleatoriamente em torno de zero, sem padrão visível.

```
residuos_treino = y_treino - y_pred_treino_v1
residuos_teste = y_teste - y_pred_teste_v1

plt.figure(figsize=(10, 5))
plt.scatter(y_pred_treino_v1, residuos_treino, color='blue', alpha=0.5, label='Treino')
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Valores Previstos')
plt.ylabel('Resíduos')
plt.title('Resíduos vs. Valores Previstos (Treino)')
plt.legend()
plt.show()

# Repita o mesmo gráfico para o conjunto de teste
```

O que observar	Interpretação
Pontos espalhados aleatoriamente em torno de y=0	✔ Bom sinal — modelo capturando bem a relação, variância constante (homoscedasticidade)
Forma de "funil" (dispersão crescente/decrescente)	⚠ Heteroscedasticidade — considere transformar variáveis (ex: log) ou usar erros-padrão robustos
Padrão curvo/sistemático	⚠ Relação não-linear não capturada — considere termos polinomiais ou outro modelo

Para ir além (opcional): este gráfico avalia **homoscedasticidade**. Para avaliar **normalidade dos resíduos** — outra suposição da regressão — o complemento natural é um gráfico Q-Q dos resíduos (`scipy.stats.probplot`).

Passo 11.1 — Salvar e reutilizar modelo + scaler

```
# Salvando
joblib.dump(scaler, 'scaler.pkl')
joblib.dump(modelo_v1, 'modelo_final.pkl')

# Carregando e usando com dados novos
scaler_final = joblib.load('scaler.pkl')
modelo_final = joblib.load('modelo_final.pkl')

novos_dados = pd.read_csv('novos_dados.csv')
novos_dados_scaled = scaler_final.transform(novos_dados)

previsao = modelo_final.predict(novos_dados_scaled)
print('Previsão:', np.round(previsao, 2))
```

Armadilha comum: os dados novos devem ter **exatamente as mesmas colunas, na mesma ordem**, usadas no treino — e sempre passar pelo mesmo scaler antes de prever. Nunca treine um novo scaler para os dados novos.

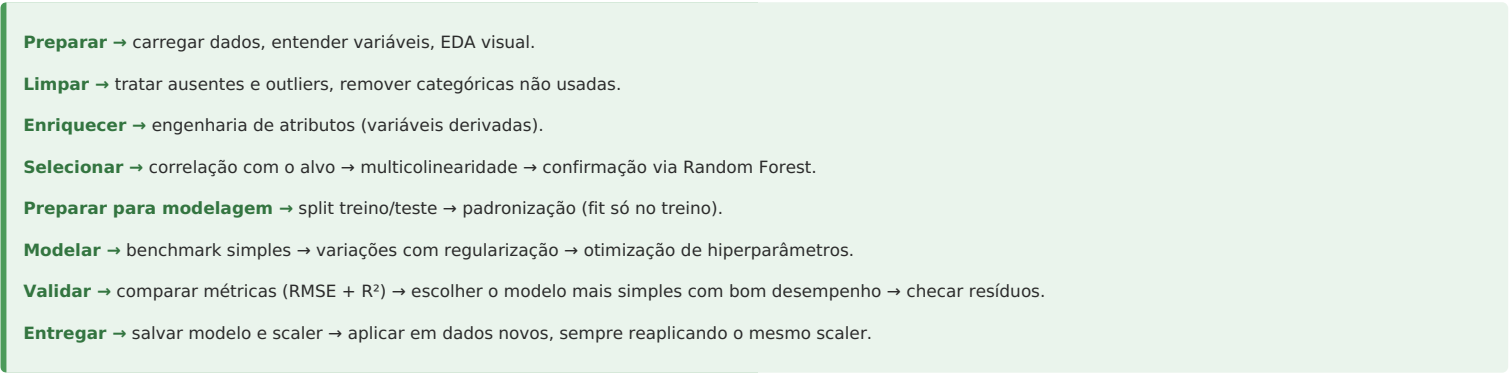
Tabela-Resumo: Comparando as 5 Versões do Modelo

Versão	Técnica	Característica-chave	Quando escolher
V1	Linear Regression	Sem regularização — benchmark simples	Quando não há sinais de overfitting nem multicolinearidade residual
V2	Lasso (L1)	Pode zerar coeficientes — seleciona variáveis	Quando suspeita que algumas variáveis são irrelevantes
V3	Ridge (L2)	Reduz coeficientes sem zerar	Quando há multicolinearidade residual entre preditoras
V4	ElasticNet	Combina L1 + L2	Quando há tanto necessidade de seleção quanto de redução de coeficientes
V5	Ridge + GridSearchCV	Otimiza o hiperparâmetro alpha via validação cruzada	Quando já escolheu a técnica (ex: Ridge) e quer o melhor ajuste fino

O que realmente importa: no projeto original, todas as versões tiveram performance muito parecida — por isso a versão mais simples (V1) foi escolhida. Isso ilustra bem o princípio da **navalha de Occam** aplicado a Machine Learning: entre modelos com desempenho equivalente, prefira o mais simples.

Mapa de Fluxo do Projeto

Visão geral de como as 11 fases se conectam



Material baseado no Projeto 3 (Data Science Academy) — use como roteiro de referência para novos datasets de Regressão Linear Múltipla.